UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
Facultad de Ingeniería en Sistemas Computacionales
Departamento de Programación de Computadoras

PROYECTO PARCIAL 2

Book&Go Lite – Gestor de Citas con Reseñas

Asignatura: Desarrollo Web / Programación Web · **Valor**: 100 puntos · **Equipos**: 4 estudiantes

Publicación: jueves 29 de octubre de 2026

Revisión preliminar: lunes 9 de noviembre de 2026

Entrega y sustentación final: jueves 12 de noviembre de 2026

1. Objetivo del proyecto

Desarrollarán en equipos una aplicación web funcional utilizando PHP, MySQL, HTML5, CSS3 y JavaScript. Su solución deberá integrar autenticación, sesiones, operaciones CRUD, consultas a una base de datos relacional y controles de ciberseguridad. Se evaluará tanto el funcionamiento como la capacidad de explicar y defender el código desarrollado.

2. Descripción general y elección del negocio

Book&Go Lite será un sistema para reservar servicios que requieren una cita. Cada equipo elegirá UN tipo de negocio y adaptará el nombre comercial, colores, imágenes, categorías y servicios a esa temática. Ejemplos: barbería, estética, tutorías, soporte técnico, fotografía o asesorías. No deben mezclar negocios sin relación entre sí.

- Definan al menos tres categorías coherentes con su negocio y registren al menos seis servicios de ejemplo distribuidos entre ellas.
- Cada servicio tendrá nombre, descripción, precio, duración y horarios disponibles.
- El CLIENTE podrá registrarse, iniciar sesión, explorar servicios, reservar, consultar o cancelar sus reservas y reseñar una atención realizada.
- El ADMIN gestionará categorías, servicios y horarios; consultará reservas y modificará sus estados.
- El sistema tendrá un menú de navegación y vistas diferenciadas para ADMIN y CLIENTE.

3. Trabajo en equipo y control de versiones

El proyecto se realizará en equipos de cuatro estudiantes. Cada integrante debe participar en el desarrollo y comprender el flujo general de la aplicación. Distribuyan tareas desde el inicio y acuerden nombres de archivos, tablas y rutas antes de programar.

Responsabilidad sugerida	Actividades
Autenticación y sesiones	Registro, inicio/cierre de sesión, permisos y protección CSRF.
Catálogo y administración	Categorías, servicios, listados, filtros y formularios CRUD.
Horarios y reservas	CRUD de horarios, disponibilidad, reservas y estados.
Reseñas, interfaz y pruebas	Reseñas, diseño adaptable, pruebas funcionales y de seguridad.

GitHub es obligatorio: creen un repositorio compartido, incorporen a los cuatro integrantes, registren avances mediante commits identificables y mantengan el código sincronizado. No se exige una cantidad fija de commits; se revisará la trazabilidad real de las contribuciones. No publiquen credenciales ni contraseñas.

4. Requisitos funcionales y reglas de negocio

4.1 Autenticación y permisos

- El registro público creará exclusivamente usuarios con rol CLIENTE.
- El ADMIN se creará mediante un procedimiento de inicialización seguro; no deben publicar su contraseña en el repositorio.
- El correo electrónico debe ser único. El sistema validará credenciales, estado del usuario y sesión activa.
- Cada rol verá un menú apropiado; además, las restricciones deben validarse en PHP, incluso al ingresar directamente la dirección de una página.
- El CLIENTE únicamente podrá consultar o modificar las reservas y reseñas que le pertenezcan.

4.2 Categorías, servicios y horarios

- ADMIN: CRUD completo de categorías, servicios y horarios (crear, consultar/listar, editar y eliminar o desactivar).
- El catálogo permitirá buscar servicios por nombre y filtrarlos por categoría.
- No se aceptarán precios negativos ni duración igual o inferior a cero.
- No se podrán crear horarios en el pasado ni con hora final anterior o igual a la inicial.
- Los horarios mostrarán fecha, hora de inicio, hora de finalización, cupos totales y cupos disponibles.
- Si un registro tiene dependencias, deberán impedir su eliminación física o implementar desactivación lógica, conservando la integridad referencial.

4.3 Reservas: flujo obligatorio

1. El CLIENTE seleccionará un servicio y un horario futuro con cupos disponibles.
2. Al crear la reserva, el sistema guardará el estado PENDIENTE y descontará un cupo de manera segura.
3. No se permitirá al mismo cliente mantener dos reservas activas para el mismo horario.
4. El ADMIN podrá cambiar PENDIENTE a CONFIRMADA y posteriormente a REALIZADA; también podrá cancelar una reserva.
5. El CLIENTE podrá cancelar una reserva propia que aún no esté REALIZADA y cuya cita no haya comenzado.
6. Cuando una reserva PENDIENTE o CONFIRMADA se cancele, se devolverá exactamente un cupo. Cancelarla nuevamente no podrá devolver cupos adicionales.

7. Una reserva REALIZADA no podrá cancelarse ni generar devoluciones de cupos.
8. La aplicación impedirá cupos negativos y cupos disponibles mayores que el total.

La comprobación de cupos y el registro de la reserva deben realizarse como una operación consistente en la base de datos (por ejemplo, mediante una transacción y actualización condicionada). No basta con deshabilitar el botón de reservar en HTML.

4.4 Reseñas

- Un CLIENTE podrá publicar una sola reseña por reserva propia con estado REALIZADA.
- La reseña tendrá calificación de 1 a 5 y comentario.
- Las reseñas se visualizarán en el detalle del servicio; no se permitirá reseñar una reserva pendiente, confirmada o cancelada.

5. Base de datos

Utilicen MySQL. El esquema mínimo tendrá SIETE tablas relacionadas: roles, usuarios, categorías, servicios, horarios, reservas y reseñas. Pueden ampliar campos si lo justifican, sin eliminar las entidades requeridas.

- Todas las tablas deben tener clave primaria; definan claves foráneas y restricciones apropiadas.
- usuarios: rol, nombre, apellido, correo único, hash de contraseña, estado y fecha de registro.
- servicios: categoría, nombre, descripción, precio, duración y estado.
- horarios: servicio, fecha, horas, cupos y estado.
- reservas: cliente, servicio/horario, estado y fechas de registro/actualización.
- reseñas: reserva, cliente, calificación, comentario y fecha; reserva única para evitar duplicados.
- Entreguen un archivo .sql importable con CREATE TABLE y datos de prueba suficientes para demostrar todas las funciones.
- El equipo debe explicar cómo sus relaciones impiden datos inconsistentes; validen que el horario seleccionado pertenezca al servicio reservado.

6. Diseño visual y experiencia de usuario

Diseñen una interfaz acorde con el negocio elegido: paleta de colores, nombre o logotipo, imágenes pertinentes, tipografía legible, menú claro, formularios comprensibles y mensajes de éxito o error. Debe poder utilizarse en computadora y teléfono.

Se permite usar Bootstrap, Tailwind, Materialize, plantillas HTML/CSS y componentes visuales con licencias compatibles. Deben adaptar los estilos a su negocio, identificar en la documentación la plantilla o biblioteca utilizada y no presentar como propio un sistema completo con PHP y base de datos ya desarrollados.

La aplicación deberá incluir las vistas necesarias para registro, login, catálogo, detalle, reservas del cliente, panel ADMIN y mantenimiento de categorías, servicios y horarios. Se evaluará la navegación funcional y no un número artificial de archivos.

7. Ciberseguridad web: implementación y evidencias

7.1 Controles obligatorios

- Contraseñas: usar password_hash() y password_verify(); nunca almacenar contraseñas en texto plano. Exigir al menos 8 caracteres.
- Sesiones: regenerar el identificador al autenticar, invalidar la sesión al salir y configurar cookies HttpOnly y SameSite; Secure cuando se utilice HTTPS.
- Intentos de login: aplicar retardo progresivo o bloqueo temporal tras cinco intentos fallidos, sin revelar si el correo existe.
- SQL Injection: utilizar PDO y consultas preparadas con parámetros en todos los valores recibidos. Los nombres dinámicos de columnas/tablas, si los hubiera, deberán controlarse mediante listas permitidas.
- XSS: validar entradas y codificar adecuadamente la salida HTML mediante htmlspecialchars() con ENT_QUOTES y UTF-8.
- CSRF: incorporar y verificar tokens en operaciones que cambien información, incluidas cancelaciones y cambios de estado.
- Autorización: verificar en el servidor el rol y la propiedad de cada recurso solicitado.
- Cabeceras HTTP: implementar Content-Security-Policy, X-Content-Type-Options y X-Frame-Options (o protección equivalente mediante frame-ancestors).
- Errores: mostrar mensajes comprensibles sin exponer consultas SQL, rutas internas ni contraseñas.

7.2 Pruebas que deberán demostrar

Prueba	Resultado esperado
SQL Injection en login o búsqueda	La entrada no altera la consulta ni evita la autenticación.
XSS en comentario o campo visible	El contenido se muestra sin ejecutar scripts.
Acceso directo CLIENTE a página ADMIN	Acceso denegado por validación PHP.
Acceso a reserva de otro cliente	No se muestra ni se permite modificar.
Formulario sin token CSRF válido	Operación rechazada sin cambiar datos.
Cinco intentos de login fallidos	Se activa el bloqueo temporal o retardo definido.
Inspección de cabeceras HTTP	Se observan las cabeceras de seguridad configuradas.

Realicen estas pruebas únicamente en su aplicación de desarrollo. Conserven capturas en security/evidencias y expliquen cada prueba en security/SECURITY.md: objetivo, pasos, resultado observado y control aplicado. No es suficiente mostrar código sin comprobar su efecto.

8. Revisión preliminar – lunes 9 de noviembre de 2026

En esta fecha el grupo mostrará presencialmente un avance funcional. No se aceptará únicamente un diseño en diapositivas.

- Repositorio GitHub creado y cuatro integrantes incorporados.
- Base de datos con las siete tablas y relaciones, importable desde SQL.
- Registro, inicio de sesión y cierre de sesión funcionando.
- Navegación diferenciada por ADMIN y CLIENTE.
- CRUD funcional de categorías y servicios, con datos de prueba.
- Interfaz base y catálogo navegable; mostrar cómo se organizaron para completar reservas y seguridad.

Valor: 15 puntos. La revisión se calificará según lo que efectivamente funcione y pueda demostrarse en la fecha indicada.

9. Entrega y sustentación final – jueves 12 de noviembre de 2026

El sistema debe estar integrado y ejecutarse en el entorno local del equipo. Se comprobarán los requisitos de los apartados 4 a 7, incluida la secuencia completa de una reserva y sus reglas de negocio.

9.1 Archivos obligatorios

- Código fuente completo en un archivo comprimido llamado PW2026_GrupoX_BookGoLite.zip.
- Script SQL de creación y datos de prueba; instrucciones claras para importarlo y ejecutar el sistema en README.md.
- Documento **PDF** con portada, integrantes, negocio elegido, diagrama entidad-relación, módulos, capturas, controles de seguridad, distribución del trabajo y reflexión individual de cada integrante.
- Carpeta security con SECURITY.md y evidencias de las pruebas indicadas.
- Referencia al repositorio GitHub y al historial de contribuciones.

9.2 Demostración presencial

Cada integrante participará en la demostración y podrá recibir una pregunta técnica individual. El profesor podrá solicitar que expliquen una consulta preparada, una relación entre tablas, un control de permisos o una validación. Preparen usuarios de prueba ADMIN y CLIENTE y datos que permitan ejecutar el flujo completo sin depender de servicios externos.

10. Rúbrica de evaluación (100 puntos)

Criterio	Evidencia principal	Puntos
Revisión preliminar	BD, autenticación, roles, CRUD inicial y navegación	15
Funcionalidad	CRUD, búsquedas, horarios, reservas, cupos y reseñas	35
Ciberseguridad	Controles implementados y pruebas verificables	25
Diseño y experiencia de usuario	Identidad visual, coherencia, navegación y adaptación	10
GitHub y trabajo colaborativo	Historial, integración y participación comprobable	5
Sustentación y dominio técnico	Demostración y respuestas individuales	10
TOTAL		100

La nota se asignará sobre evidencias funcionales. Una pantalla que solo simula resultados sin operar con la base de datos no se considerará una funcionalidad terminada. La evaluación del trabajo colaborativo y la sustentación podrá reflejar diferencias individuales cuando existan evidencias de participación desigual.

11. Uso de inteligencia artificial y honestidad académica

Pueden utilizar herramientas de IA como apoyo para comprender conceptos, investigar errores, revisar código y proponer mejoras. No deben delegar a la IA la generación íntegra del proyecto para presentarlo como propio. Si utilizan IA, documenten las herramientas, consultas relevantes, fragmentos incorporados, verificaciones y modificaciones realizadas. Cada estudiante deberá poder explicar el código entregado. Verifiquen las respuestas de IA con documentación y pruebas; su uso no exime de responsabilidad técnica.

12. Recomendaciones finales

Organicénse desde el primer día, distribuyan módulos, acuerden el modelo de datos y hagan integraciones frecuentes. Eviten dejar la seguridad, las pruebas y la unión del código para el último día. El objetivo es demostrar que pueden construir en equipo una aplicación funcional, bien presentada y con controles de seguridad verificables.